

DisasterClaw: A georeferenced platform and task benchmark for closed-loop UAV disaster inspection

Anonymous authors

Anonymous submission,

Abstract

Aerial disaster inspection requires linking sensing actions to evidence and task outcomes, yet disaster datasets are usually organized as static image records. We present DisasterClaw, a georeferenced platform and task benchmark that transforms paired xBD imagery into map-anchored interactive scenes for closed-loop UAV inspection. Disaster imagery, building annotations, task regions, and vehicle states share a geographic reference. Presence, damage, count, and spatial tasks are generated programmatically, with reference answers computed from registered annotations and geometry. Automated checks and independent review by two annotators yield 160 final tasks over 44 regions in three disaster events. An interactive console and a headless runner support traceable observation–perception–action–answer episodes. A development camera-ladder diagnostic shows that narrower footprints can both correct and harm building predictions. Seven frozen policies evaluated over three generation repeats produce 3,360 episodes. The task-conditioned policy attains 75.00% accuracy, compared with 72.29% for no reobservation, and executes 161 reobservations compared with 923 for always reobserving. The observed gain over holding is 2.71 percentage points, but the prespecified ROI-cluster, multiplicity-adjusted interval spans -2.04 to $+7.50$ points. Thus, a stable advantage over holding is not established. The contribution is a platform and benchmark for measuring task-dependent observation value, bounded by rendered orthographic imagery, simplified motion, prior xBD exposure of the fixed ChangeOS backbone, and three evaluation events.

Keywords: UAV disaster inspection, Georeferenced simulation, Task benchmark, Active perception, Closed-loop evaluation, Vision-language agents

1. Introduction

Disaster response creates a demanding setting for aerial embodied intelligence. An inspection agent must interpret a task, place its sensor over a relevant region, decide whether the current evidence is sufficient, and return a usable assessment under limited observation and motion budgets. Static remote-sensing benchmarks measure recognition on fixed images, but they do not expose the decision of where and when to observe again. General aerial navigation environments provide motion and language tasks, yet they do not necessarily connect a changed view to a disaster-specific answer. A reproducible bridge between these settings is needed to study whether an agent’s sensing actions improve the decision that motivated them.

A central challenge is how to transform static remote-sensing records into an environment in which an agent can act, reobserve, and be evaluated. Geographic metadata alone do not provide that environment. Disaster imagery, annotated buildings, task regions, vehicle states, and sensing actions must remain consistent within a shared world representation. This is particularly demanding for orthographic imagery: an apparent altitude change must respect the source-resolution ceiling, while the reference answer must remain tied to the task region even when the current view covers only part of it.

We present DisasterClaw, a georeferenced platform and task benchmark for studying task-dependent active observation in single-UAV disaster inspection. Its data-to-world pipeline places paired xBD records at their geographic footprints within a basemap and represents buildings, task ROIs, targets, and the UAV in the same coordinate system. Programmatic rules derive presence, damage, count, and spatial tasks from registered annotations and geometry. Reference answers are computed without a language model, and generated questions undergo automated checks and human review. The resulting world supports both interactive operation and headless benchmarking through shared observation, perception, action, and logging interfaces.

The distinction between a task and an observation is central to this design. Buildings are the units of damage perception, ROIs define the scope of questions, executed actions consume observation budgets, and complete episodes are scored by terminal answers. The agent can search, center on a target, descend to obtain a narrower footprint, answer, or abstain. Each executed view change is recorded together with the resulting evidence and answer. Generation variability is recorded separately because a vision-language model can change its answer even when environmental evidence is unchanged.

DisasterClaw is used here to examine budgeted reobservation. A closer view allocates more source pixels to the target region while discarding some

surrounding context. Whether this helps depends on the task: a building label, a count, and a spatial relation require different evidence. We first characterize the ChangeOS binary perception response across three footprints on a development population. The main experiment compares seven frozen policies on a separate set of 160 questions reviewed independently by two annotators, scoring complete answers over three generation repeats.

Recent aerial embodied benchmarks emphasize broad skill coverage, realistic navigation, or standardized perception–decision–action evaluation, while recent remote-sensing agent benchmarks emphasize multimodal reasoning and tool use over large image collections. DisasterClaw targets a narrower complementary question: given a georeferenced disaster scene and a task-specific evidence requirement, can an executed observation action be traced to a change in perception evidence and ultimately to a corrected or harmed terminal answer? The contribution is therefore not broader environment realism or a new foundation model, but a controlled measurement interface for task-dependent observation value in disaster inspection.

The contributions are as follows.

1. We develop a map-anchored disaster-world construction pipeline that converts paired xBD imagery and building annotations into georeferenced interactive scenes. Task ROIs, vehicle states, targets, and movement actions share a geographic reference, enabling repeatable observation updates.
2. We construct an ROI-scoped disaster-inspection benchmark with four task families. Reference answers are derived deterministically from building labels and geometry. Automated schema, answer, geometry, duplicate, and split checks precede independent review by two annotators and adjudication.
3. We implement a traceable closed-loop inspection platform with an operator console and a headless runner. Vehicle states, observations, perception evidence, decisions, executed actions, budgets, and terminal answers connect each sensing decision to its observed task outcome.
4. We evaluate task-dependent active observation on 160 verified tasks and 3,360 policy episodes. T1_TASK has the highest observed accuracy and the fewest reobservations among active policies in this sample, while adjusted intervals leave its advantage over holding and raw entropy uncertain. The results expose both benefits and harms of additional observations.

The present platform is an imagery-based research environment, not a six-degree-of-freedom flight simulator. It uses orthographic source imagery and simplified motion, and it does not model wind, aircraft dynamics, occlusion,

collision avoidance, or battery discharge. These boundaries determine the scope of every experimental claim.

2. Related work

2.1. Aerial embodied agents and simulation platforms

Vision-and-language navigation connects natural-language instructions to sequential visual decisions [1]. AerialVLN extends this problem to UAV navigation, while CityNav introduces large-scale real-world aerial navigation data [2, 3]. OpenFly provides a broader aerial vision-language platform, toolchain, and trajectories across multiple heights and scenes [4]. These works establish language-conditioned aerial motion and height-varying observation as active research areas. DisasterClaw addresses a complementary requirement: constructing task-scoped, paired disaster observations and measuring whether an inspection action changes the requested disaster assessment.

Large pretrained models can be composed with perception and navigation modules without treating the entire stack as a newly learned end-to-end policy [5]. CityNavAgent uses hierarchical semantic planning and global memory, spatial text representations support language-model reasoning in aerial navigation, and AerialClaw exposes modular aerial skills and feedback [6, 7, 8]. DisasterClaw adopts this modular view. Semantic maps, planning, memory, and tool execution support the implemented agent; the platform contribution lies in their integration with georeferenced disaster scenes, controllable inspection observations, and a common measurement interface.

CityEQA connects active urban exploration to embodied question answering and evaluates answers after view refinement [9]. Its setting is a direct precedent for active aerial question answering. DisasterClaw differs in task domain, source imagery, action abstraction, and evaluation target. Rather than benchmarking general-purpose flight skills or high-fidelity navigation, it couples a georeferenced disaster scene to a controlled observation transition and measures whether that transition changes task-relevant evidence and the terminal inspection answer. This narrower scope motivates reporting observation construction, action execution, and task-level correction or harm with equal precision.

Recent work has expanded aerial embodied intelligence beyond instruction-following navigation. AeroVerse organizes UAV-agent evaluation around scene awareness, spatial reasoning, navigational exploration, task planning, and motion decision, providing a broad benchmark suite for aerospace embodied world models [10]. BEDI further formalizes UAV embodied-agent evaluation through a perception–decision–action chain and evaluates semantic and spatial perception, motion control, tool use, planning, and action generation across

real and virtual settings [11]. OpenUAV and the UAV-Need-Help benchmark emphasize realistic aerial trajectory execution and assistant-guided object search in three-dimensional environments [12]. These platforms address general aerial embodiment, realistic navigation, and standardized skill evaluation. DisasterClaw is complementary rather than competing in simulator fidelity: it uses georeferenced pre-/post-disaster imagery to make the task value of a controlled view change directly observable at the final disaster-inspection answer.

2.2. *Disaster interpretation from remote-sensing imagery*

xBD provides paired satellite images and building-damage annotations for disaster assessment [13]. RescueADI studies autonomous tool use for adaptive interpretation of remote-sensing disaster images [14]. Thus, paired damage recognition and agentic disaster analysis both predate this work. Recent remote-sensing resources further broaden the evaluation of multimodal reasoning and agentic analysis. ThinkGeo evaluates tool-augmented language-model agents on multi-step remote-sensing tasks, including disaster assessment and change analysis, and scores both intermediate tool execution and final answers [15]. DisasterM3 provides a large-scale multi-hazard, multi-sensor vision-language benchmark for disaster damage assessment and response, covering perception and reasoning across diverse disaster events [16]. These resources strengthen static-image reasoning, multimodal generalization, and structured tool use. DisasterClaw addresses a different axis: the observation itself is an environment variable, so the agent can execute a view-changing action and the benchmark can measure whether the newly acquired evidence improves or harms the final task answer. DisasterClaw uses such static records as the substrate of an interactive inspection environment: a geographic window becomes an observation, vehicle state changes the window, and the resulting episode is scored at both building and task levels.

The distinction is a data-to-world construction route: registered imagery defines an observation environment, while source annotations and geographic rules define the task benchmark. This is complementary to the navigation, tool-use, and general embodiment objectives above, not a claim that all other platforms use only synthetic scenes. The transformation remains constrained by the data. Cropping and resampling an orthographic product can alter the number of pixels assigned to a target, but it cannot synthesize unrecorded viewpoints or detail below the native source scale. The platform is consequently suited to controlled studies of spatial coverage, observation allocation, and answer formation. Evidence from it cannot be presented as a measurement of real UAV image formation or field performance.

2.3. Active perception, uncertainty, and task value

Active perception treats sensing as an action selected in service of a task [17]. Planning under partial observability further connects information gathering with uncertain states and action outcomes [18]. This perspective motivates the central platform requirement: a new observation should be evaluated by the decision it enables and the cost it incurs.

Entropy reduction is a convenient trigger but is not equivalent to expected task utility. A distribution can become more concentrated around an incorrect class, while a view may help a count or spatial relation without changing the most uncertain building label. DisasterClaw therefore records classification, correction and harm, final question correctness, and executed observation count as separate endpoints.

Temperature scaling addresses probability calibration, while evaluation under dataset shift shows why in-distribution calibration should not be treated as a universal reliability property [19, 20]. Selective prediction measures the trade-off between coverage and error, and uncertainty-aware language-model planning has been studied through help-seeking behavior [21, 22]. We use these distinctions to separate raw and calibrated binary-damage triggers, prediction-set sensing rules, and the VLM’s generated answer. None alone establishes reliability for the composed agent.

3. The DisasterClaw platform

3.1. Design objectives and system architecture

DisasterClaw is an imagery-based simulation and evaluation framework for single-UAV disaster inspection. Its purpose is to turn georeferenced disaster records into closed-loop episodes in which an agent changes its map position or observation footprint, receives a new image, and is scored on a disaster task. The implementation has four connected layers (Fig. 1): a task interface, a disaster-response agent, a simulated geospatial world, and an instrumentation layer. The operator-facing path uses a React/Vite console connected to a Flask/Socket.IO service. Quantitative experiments use a headless runner that calls the same environment and controller modules without rendering the user interface.

Table 1 states what each layer implements and the boundary attached to it. This distinction is central to the intended use of the platform. Its geospatial state and image renderer support controlled observation studies; they do not reproduce the complete physics of an aircraft or camera.

The design has three objectives: retain the geographic relationship between source imagery and inspection targets, expose a controllable observation

Table 1: Implemented DisasterClaw capabilities and their evaluation boundaries.

Capability	Implemented function	Boundary in this study
Scene	Georeferenced pre-/post-disaster tiles, building annotations, map anchors, target ROIs, and contextual basemap	Three evaluated events and 44 ROIs; incomplete source archive and no global coverage claim
Observation	Nadir fixed-FOV crop and resampling at three footprint widths; paired archival and current-image channels	Orthographic imagery; no new viewpoint, acquisition time, or detail below the assumed 0.5 m/pixel source scale
Vehicle and actions	Geographic and local position, altitude, heading, speed, task state, straight-line motion, centering, descent, hover, marking, and reporting	One UAV with simplified kinematics; no six-degree-of-freedom dynamics, wind, collision, communication, or battery model
Perception and models	Building localization and binary no-damage/damaged evidence, geographic cross-view evidence memory, task-specific spatial summaries, and Qwen answer generation	Fixed ChangeOS backend with prior xBD exposure; fixed Qwen2.5-VL-7B-Instruct answer model; no claim of event-disjoint perception generalization
Tasks	Operator instructions, damage/presence/count/spatial questions, and archived language-guided navigation instructions	Task-scoped simulation; no claim of unrestricted natural-language or open-world competence
Logging and evaluation	Interactive monitoring, headless episodes, paired task IDs, intermediate evidence, actions, budgets, answers, and terminal scores	All 3,360 final online episodes are covered by local shard-level consistency audits; complete public release packaging remains pending
Physical boundary	Map-anchored scene transitions and a controlled observation ladder	No photorealistic sensor synthesis, aircraft certification model, hardware-in-the-loop result, or physical-flight validation



Figure 1: Data-to-episode workflow in DisasterClaw. Paired xBD imagery and building annotations provide a geographic scene and reference evidence; map anchoring makes observations depend on vehicle state; task generation, checks, and review produce a frozen benchmark; the platform records closed-loop inspection episodes. Image panels illustrate these stages rather than documenting one final-test trajectory.

through a common agent interface, and preserve the evidence needed to reconstruct task outcomes. The benchmark is constructed from the same registered source records, but its reference annotations are kept separate from agent observations. This separates world construction, answer construction, and policy execution.

3.2. Map-anchored disaster-world construction

The scene layer reads paired pre- and post-disaster tiles, building polygons, damage labels, and geographic metadata derived from xBD. Pixel-to-geographic and geographic-to-pixel affine transforms place each image on its recorded ground footprint. When a tile becomes active, its center defines the local world anchor. Robot, target, and report coordinates remain available as latitude, longitude, and altitude, while local north and east offsets are recomputed relative to that anchor.

The interactive map can place the active disaster image over a satellite or street basemap, display the tile boundary, and color annotated buildings for operator reference. These annotation overlays are not passed to the evaluated agent as visual input. In rendered observations, georeferenced disaster tiles supply task evidence and the surrounding basemap supplies spatial context when a wider footprint extends beyond the available disaster tile. Reference answers and building-level scores remain restricted to the annotated target ROI, so contextual background is not treated as disaster ground truth.

The evaluated scene population contains three disaster events and 44 target ROIs. This is the experimental subset, not a claim that the software can reproduce every disaster represented by the source dataset. Missing

tiles, sparse geographic coverage, affine-registration error, and differences in acquisition date all constrain the resulting environment.

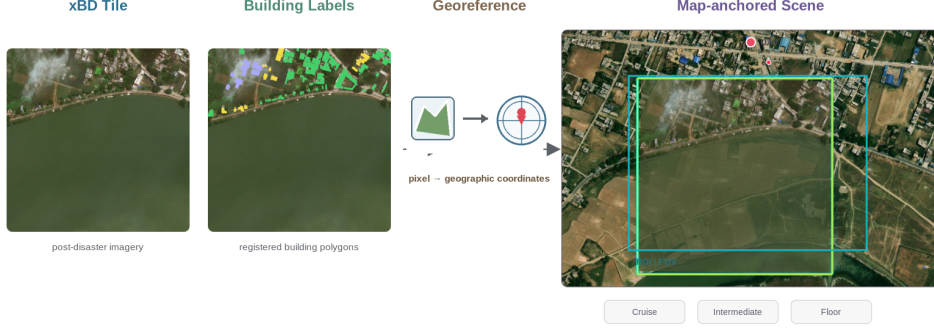


Figure 2: Geographic embedding of disaster records. Image footprints and building polygons are placed in a shared map coordinate system, allowing the environment to associate observations and target locations with geographic state. The supplied visualization illustrates registration and overlays; its colors and markers are not a verified final-task answer key. Basemap context outside the annotated ROI does not supply disaster ground truth.

3.3. Controllable observation model

The simulated sensor is nadir-facing, with horizontal field of view θ and fixed output width W . For vehicle height h , the ground-footprint width L and effective ground sampling distance g are

$$L(h) = 2h \tan(\theta/2), \quad g(h) = \frac{L(h)}{W}. \quad (1)$$

We use $\theta = 60^\circ$, $W = 1024$, and an assumed source scale of 0.5 m/pixel. A 1024-pixel source tile therefore spans 512 m. Footprints of three, two, and one tile widths define cruise, intermediate, and floor states at approximately 1330.2, 886.8, and 443.4 m, with effective GSDs of 1.5, 1.0, and 0.5 m/pixel (Fig. 3). The heights are derived parameters of this camera abstraction; they are not measured flight altitudes from the source imagery.

For paired damage perception, the post-disaster channel is sampled at the current footprint and fixed output size. The corresponding pre-disaster window is retained at the assumed source scale before the pair is brought to the detector input geometry. This represents access to an archival reference and a controllable current observation. It also means the two channels undergo different resampling. The platform records this choice because it can contribute to any observed view response.

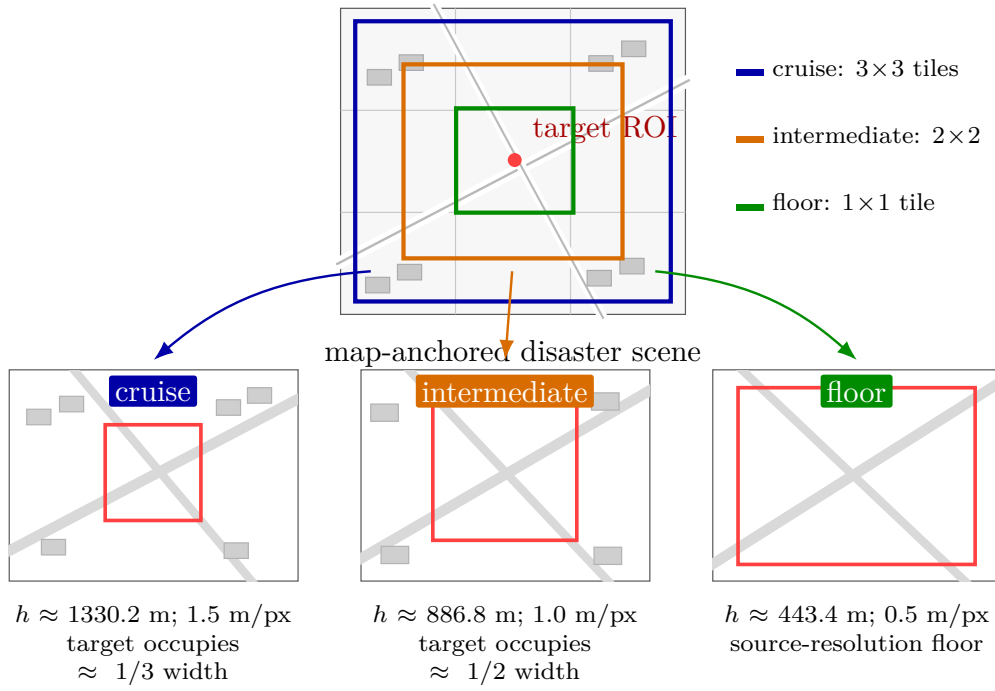


Figure 3: Observation construction from one georeferenced scene. Each state is rendered to 1024×1024 pixels. Descending narrows the ground footprint and assigns more existing source pixels to the target ROI; the floor state reaches, but cannot exceed, the assumed source resolution.

Changing height only crops and resamples existing orthographic imagery. The renderer does not create sub-source detail, oblique views, three-dimensional parallax, self-occlusion, atmospheric effects, motion blur, or a new acquisition time. At the floor, further descent cannot add information under the adopted source model.

3.4. Agent–environment interaction interface

The world model stores a single UAV with geographic position, altitude, heading, speed, flight status, and task state. It also stores targets, reports, the active disaster tile, map bounds, and timestamps. Motion is implemented by straight-line interpolation in a locally anchored horizontal frame. A position update is converted between geographic coordinates and local north–east offsets, and altitude is recorded as the vertical state.

The task planner and closed-loop agent share a finite set of environment tools: fly to a geographic coordinate, move by a relative horizontal or vertical offset, hover, run disaster detection, mark a target, and produce a report. The inspection controller additionally uses answer, abstain, and stop as episode decisions. A reobservation maps to one or more movement primitives, such as centering on evidence and descending by one camera-ladder state, followed by a new render and perception pass.

This action model supports repeatable studies of where the sensor is pointed and how often another observation is requested. It omits rigid-body attitude dynamics, acceleration limits, wind, terrain following, collision checking, communications, and battery discharge. Executed action count is therefore the principal online cost in this paper. Any time estimate obtained from configured speeds remains a simulator quantity rather than measured endurance.

3.5. Instrumentation and benchmark execution

The console exposes disaster selection, a georeferenced situation map, manual flight controls, language-task submission, perception displays, agent state, and event logs. An operator can activate a disaster tile, select a target location, issue an inspection task, observe the planned or closed-loop actions, and inspect the returned damage evidence. The operator console supports qualitative inspection of the same scene state, perception evidence, decisions, actions, budgets, and event logs used by the headless benchmark. The interface is intended for monitoring and diagnosis; all quantitative results in this study are produced by the frozen headless protocol.

The headless benchmark initializes a question and vehicle state, invokes the same observation and action modules, and writes the evolving answer, evidence, controller decision, movement, budget, and terminal outcome. Frozen

question identifiers allow policies to be compared on paired tasks. The implementation also supports a fixed-view mode that holds the environment state constant while repeating answer generation. The final benchmark binds generation seeds to question, repeat, step, and call role, making environmental changes and subsequent generation calls separately traceable. Policies may still make different numbers of later calls, so whole-policy comparisons are not pure action-only effects.

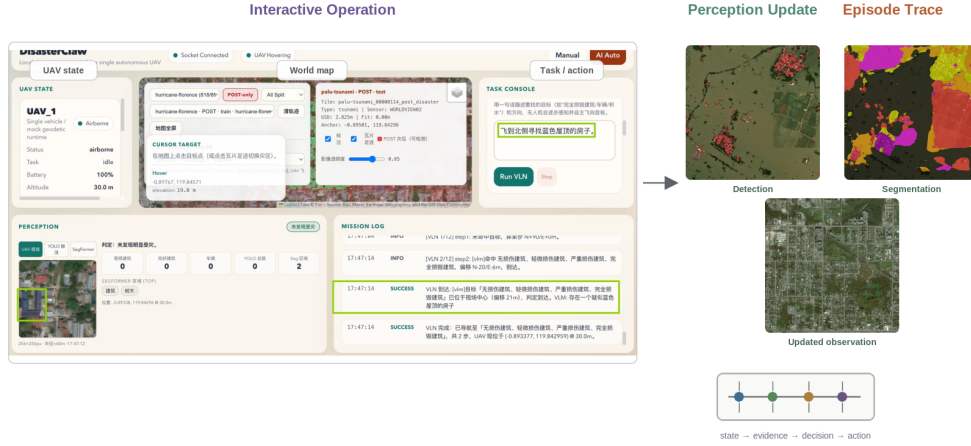


Figure 4: Representative console, perception displays, and closed-loop instrumentation. This supplied composite shows general platform interfaces, including legacy YOLO/SegFormer and VLN controls; it is not a single synchronized episode or a screenshot of the frozen ChangeOS experiment. The displayed 30 m setting is an interface example, not a height used in the camera-ladder evaluation. Quantitative results use the headless protocol described in Section 6.

4. DisasterClaw benchmark construction

The benchmark converts registered disaster records into questions whose answers are computed from annotations and geometry before any agent is evaluated. A task is not simply an image-text pair: it binds a question to a geographic ROI, an initial vehicle state, an answer vocabulary, and a reference answer that remains fixed as the agent moves. Table 2 summarizes the final set and distinguishes its size from the number of executed episodes.

4.1. Source records and candidate ROI selection

Candidate records require a post-disaster image, a usable geographic footprint, and the corresponding label file. Paired pre-disaster imagery supplies the archival reference used by the damage-perception pipeline. The

Table 2: Final benchmark and execution scope. Repeated policy runs do not increase the number of independent questions or source regions.

Item	Specification
Source evidence	Paired xBD imagery, registered building polygons, and damage labels
Final task set	160 questions over 44 geographic ROIs from three events
Event composition	Hurricane Michael: 48; Moore tornado: 51; Nepal flooding: 61 questions
Task families	Presence, damage, count, and spatial relation
Reference answers	Deterministic annotation and geometry rules; no LLM-generated ground truth
Label space	Binary no-damage/damaged; count buckets 0, 1, 2, 3+; eight compass directions
Quality control	Coverage and split checks, schema/answer checks, duplicate detection, ambiguity flags
Human review	Two independent reviews and disagreement adjudication before freezing
Online evaluation	Seven policies $\times$ 160 questions $\times$ three repeats = 3,360 episodes

ROI index evaluates whether available georeferenced post-disaster tiles cover the configured cruise observation footprint. The generator’s default minimum coverage is 0.80: this is the fraction of that observation footprint supported by disaster imagery, not a requirement that a task ROI contain 80% buildings or that 80% of the ROI be damaged.

Valid building records provide a polygon, a geographic centroid, a source identifier, and one of the four supported xBD damage labels. For the binary benchmark, minor damage, major damage, and destroyed are mapped to damaged; no-damage is retained as the other class. Unclassified or unusable records do not define binary reference answers. Source polygons and damage labels are used for task construction and scoring, not shown as ground-truth overlays to the evaluated agent.

4.2. Event split and geographic ROI construction

The generator selects eligible records from the configured evaluation events, excludes tile identifiers already recorded in the consumed-ROI registry, and uses seeded shuffling with round-robin selection across events. A tile can yield up to four task families subject to their prerequisites. Consequently, the final 160 questions are neither 160 independent tiles nor necessarily 40 questions

per family. The observed event counts are reported in Table 2 rather than described as an exactly balanced allocation.

Each question stores its source tile identifier, geographic ROI bounds and center, and initial latitude, longitude, and altitude. A building belongs to the reference ROI when its centroid lies within the closed ROI bounds. Let B_R be this building set and $D_R \subseteq B_R$ its damaged subset. Reference answers are defined on this fixed geographic region, not on whichever crop happens to be visible at a later step. The consumed-record check separates the final task ROIs from development use; it does not establish event-disjoint pretraining for the fixed ChangeOS checkpoint.

4.3. Programmatic task and answer generation

Questions are instantiated from task templates, while reference answers follow explicit rules:

- **Presence:** whether a damaged building exists in the ROI, with answer $\mathbf{1}[|D_R| > 0]$. Positive evidence can settle an existence question, whereas a negative answer requires adequate regional coverage.
- **Damage:** whether a designated building is damaged. A valid source building is selected, its identifier and location define the target, and its binary annotation determines the answer. The visible target cue identifies what to inspect; it does not expose the damage label.
- **Count:** the number of damaged buildings in the ROI, mapped to 0, 1, 2, or 3+. This requires maintaining the regional extent and avoiding duplicate counting across views.
- **Spatial:** the compass direction from the ROI center to the nearest damaged building. Distance is computed in a local metric north–east frame; bearing is quantized into eight 45-degree sectors. This task requires a nonempty D_R and preserves an explicit geographic reference point.

No language model supplies the ground-truth answer. The task schema also retains answer choices, target metadata when applicable, and geometric difficulty or ambiguity indicators. Such indicators describe construction conditions, such as nearby candidates or boundary proximity; they are not measurements of agent difficulty or evidence of balanced difficulty strata.

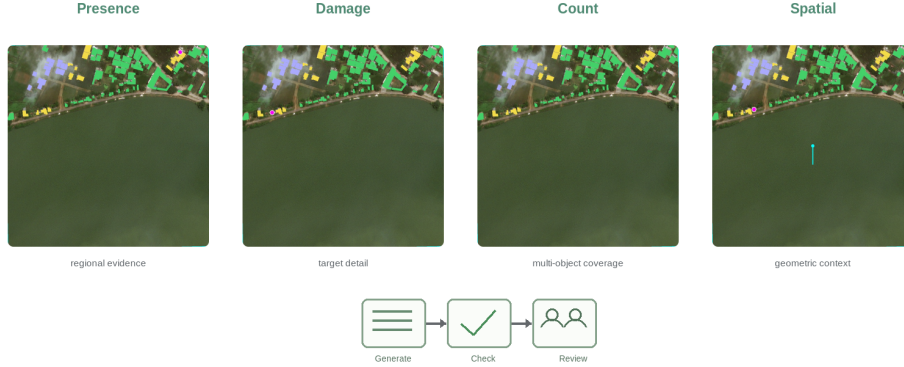


Figure 5: Illustrative spatial evidence for the four task families. Presence concerns existence within a region, damage concerns a designated building, count concerns region-wide aggregation, and spatial questions concern a target’s direction relative to a geographic reference. These supplied panels illustrate task construction; they do not provide verified question–answer pairs from the frozen 160-question set. Colored polygons and markers are construction overlays, not ground-truth inputs to the agent.

4.4. Automated quality control

Generation and review checks cover required fields and answer vocabularies, geographic validity, consistency of answers with annotations and geometry, the binary label mode, event membership, and previously consumed records. Duplicate checks use the source tile, task type, question text, and answer. Geometric ambiguity notices are retained for review rather than interpreted as an automatic guarantee that a question is visually answerable. Records that fail acceptance checks require correction and renewed validation before inclusion in a frozen evaluation set.

The runtime task context contains the permitted question, ROI, initial state, and target cue. Reference labels and answer fields remain in the evaluation path. This separation matters because the same source annotations support both world inspection by an operator and benchmark scoring; displaying an overlay in the console does not authorize providing it as agent evidence.

4.5. Dual review and benchmark freezing

Two annotators independently reviewed the 160 final questions for task type, ROI, target identity where applicable, reference-answer consistency, and answerability from the registered imagery and annotations. Disagreements were resolved before freezing; ambiguity notices were retained in the audit trail. Human review checks the applicability and clarity of the programmatic reference rather than replacing it with a language-model answer.

The final task set is distinct from the 160-question development set used to select policy rules. Once frozen, question identifiers and answers are shared across all policies and repeats. The execution protocol records task and review inputs together with source, prompt, checkpoint, and configuration identities. The locally available protocol and the remaining review-artifact release requirements are distinguished in Appendix A. Neither dual review nor deterministic answers remove all annotation error or language-template shortcuts; these remain limitations of the benchmark.

5. Task-conditioned active observation

5.1. Task, state, and evidence

Each inspection episode has a question q , a geographic region of interest (ROI), a task type (presence, damage, count, or spatial relation), and a reference answer y_q . At step t , the simplified vehicle state $s_t = (x_t, y_t, h_t)$ determines a rendered image $I_t = \mathcal{R}(\mathcal{M}, s_t)$ from scene $\mathcal{M}$. The agent also receives structured evidence z_t from the fixed perception backend and registered environment state. It may search, request a task-appropriate reobservation, answer, abstain, or stop. A reobservation is counted only when a movement executes and a new view is rendered; a repeated answer on an unchanged view is not an observation action.

Figure 6 summarizes the agent loop. The benchmark retains the intermediate answer, decision, movement, evidence, generation-call keys, and terminal outcome. It scores the terminal normalized answer against y_q , with abstentions counted as unsuccessful answers. This is an episode-level question-answering endpoint rather than an average of building classifications.

5.2. Fixed binary perception and cross-view memory

The main experiments use the same frozen ChangeOS ResNet-34 checkpoint for every policy. It detects buildings in registered pre-/post-disaster imagery and supplies a binary no-damage/damaged probability vector. ChangeOS is an external perception tool, not a newly trained contribution of this study. Its checkpoint has prior xBD exposure; neither the perception evaluation nor the online results demonstrate transfer of the detector to unseen disaster events.

For a binary vector $p_i = (p_{i0}, p_{i1})$, the damage-uncertainty score is

$$H_2(p_i) = -\frac{1}{\log 2} \sum_{j=0}^1 p_{ij} \log p_{ij}. \quad (2)$$

The raw-entropy policy uses the uncalibrated binary probabilities, whereas calibrated entropy and a prediction-set trigger are supplemental comparisons.

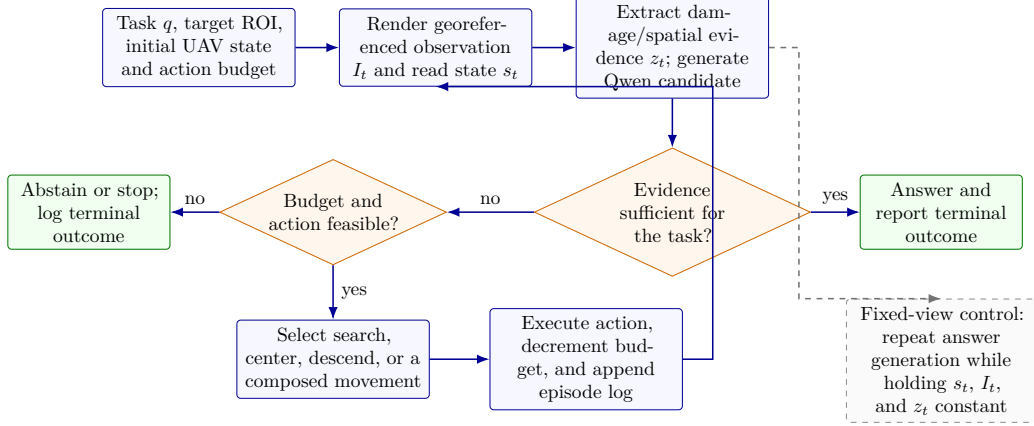


Figure 6: Closed-loop inspection state machine. Feasibility and budget checks distinguish a requested reobservation from an executed view change. The unchanged-view branch is available for diagnosis but is not counted as reobservation.

Historical four-class temperature and APS thresholds are not transferred to this experiment. For count tasks, the task-conditioned gate estimates uncertainty over the answer buckets 0, 1, 2, 3+ from the binary building probabilities; for spatial tasks it uses registered target geometry and a GSD-dependent bearing uncertainty. Thus a single uncertain building label is not treated as a sufficient reason to crop away a complete count or spatial ROI.

The online evidence memory matches predicted objects by geographic proximity and combines observations across views, preserving the history of observation IDs and ROI coverage. This avoids treating each new crop as an unrelated scene or automatically discarding earlier wide-view evidence. Answer generation uses a locally loaded Qwen2.5-VL-7B-Instruct model with the task image and selected structured state; the evaluated configurations use the same hybrid answer path. The model’s output is normalized to the task’s answer vocabulary before scoring. A deterministic rule answerer and pure-VLM paths exist as development diagnostics, but are not pooled into the seven-policy final comparison.

5.3. Task-conditioned observation gate

The task-conditioned policy (T1_TASK) estimates whether an executable next view is worth its cost. In its implemented rule, the candidate utility is

$$U_q = u_q(1 - h_{t+1}/h_t) - 0.05 \hat{T}_{\text{move}}/60 - (1 - \hat{C}_{\text{ROI}}), \quad (3)$$

where u_q is task-specific uncertainty, $\hat{T}_{\text{move}}$ is a configured-speed motion-time estimate in seconds, and $\hat{C}_{\text{ROI}}$ is predicted ROI coverage after the candidate

action. This is a fixed heuristic, not a learned or calibrated estimate of expected task accuracy. The frozen gate requires $u_q \geq 0.5$ and $U_q \geq 0.05$; count and spatial tasks additionally require predicted coverage of at least 0.98. A damage question may center on a visible marked target and descend. Count and spatial questions retain the geographic center and descend only if the task ROI remains covered. Presence positives are answered from the wide view, while missing evidence uses the search path. All thresholds were chosen before the final run using development evidence.

5.4. *Comparator policies and action budgets*

The four primary comparators are A0_HOLD (search allowed, no reobservation), A1_RANDOM (independent seeded random trigger), A2_ALWAYS (request whenever feasible), and A3U_RAW_ENTROPY (binary raw-entropy trigger). A3_ENTROPY and A4_CONFORMAL are supplemental. Each active policy has the same exogenous cap of two reobservations per episode, and all configurations permit up to six search steps. A0_HOLD has a zero reobservation cap. The cap is shared, but realized action counts differ because policy requests and feasibility differ; the seven outcomes are operating points, not a randomized budget sweep.

6. Experimental protocol

The study asks five linked questions: how observation scale changes building-level evidence (RQ1); how the frozen policies differ in terminal task accuracy (RQ2); what action costs and answer transitions accompany those outcomes (RQ3); how outcomes vary with task evidence requirements (RQ4); and which bottlenecks and inference limits remain (RQ5). These questions evaluate the platform’s observation–evidence–answer chain, rather than treating navigation realism or universal policy superiority as the endpoint.

6.1. *Development diagnostics and frozen final tasks*

The ChangeOS camera-ladder diagnostic uses 43 development ROIs containing 7,238 annotated buildings. It compares cruise, intermediate, and floor predictions on the same ground-truth building identities. The full-ground-truth endpoint counts an unmatched building as incorrect; matched-only classification and probability metrics use only matched detections. These are development and mechanism measurements, not final policy scores.

The confirmatory online evaluation uses the frozen 160-question, 44-ROI benchmark constructed in Section 4, separate from the 160-question development set used to choose policy rules. All policies receive the same final questions, initial states, and reference-answer definitions. Dataset construction

and review are not repeated for each policy or generation repeat. Execution identities and the provenance boundary are documented in Appendix A.

6.2. Online execution and endpoints

Seven policies run on each of the 160 final questions for each of three generation repeats, producing 3,360 planned episodes. Four shards partition each repeat without changing the question set. The action base seed is 42; generation calls derive their seed from base 42000 and (qid, repeat, step, call role). Qwen sampling settings, source fingerprint, prompt hash, and ChangeOS weights are frozen in the protocol. All 12 shard audits passed with zero execution errors. Repeats provide three generation conditions on the same 160 questions; they are not new independent questions, ROIs, or events.

The prespecified primary endpoint is terminal task accuracy over all questions, averaged over repeats. Abstentions count as incorrect for this endpoint. Secondary descriptions include accuracy by event and task type, abstention, within-policy answer correction and harm, executed reobservations, state-derived horizontal/vertical motion, and episode wall time. A correction or harm observed inside one policy’s trajectory is not the counterfactual effect of its action relative to another policy. Later VLM call counts can also differ across policies, so paired accuracy contrasts estimate whole-policy outcomes rather than a pure image-change effect.

Four primary contrasts were declared before execution: T1_TASK against holding, random, always reobserving, and raw entropy. Pairing uses the same (repeat, qid) units. For each contrast, a 5,000-draw ROI-cluster bootstrap resamples all questions and three repeats within each sampled ROI, with seed 42. The four primary comparisons use Bonferroni-adjusted 98.75% percentile intervals; 95% intervals are descriptive. Only three events are represented, so these intervals characterize variation among the observed ROI clusters, not transfer to unseen disasters. Event and question-type breakdowns are descriptive rather than additional confirmatory tests.

Executed reobservation count is the principal observed action-cost measure. Episode wall time is measured software runtime for this execution, not UAV flight time. Trajectory-state displacements are geometric estimates, and configured-speed d/v values are estimates rather than measured airborne duration. No randomized common-cap sweep or component-level render/ChangeOS/controller/Qwen timing is available. Accordingly, accuracy and cost are reported as realized policy operating points.

7. Results and discussion

7.1. RQ1: A narrower observation changes both detection and correctness

The development camera-ladder diagnostic keeps 7,238 ground-truth buildings in 43 ROIs fixed while changing the rendered footprint. ChangeOS localization recall rises from 22.09% at cruise to 34.06% at intermediate and 56.52% at floor. When unmatched ground-truth buildings are counted as incorrect, binary accuracy rises from 20.01% to 31.06% and 53.65%, respectively. Among matched buildings alone, accuracy is 90.56%, 91.20%, and 94.92%, with macro-F1 of 0.8714, 0.8743, and 0.9232. Matched-only Brier score falls from 0.0657 to 0.0384 between cruise and floor; these probability metrics are not defined over unmatched ground-truth buildings. The large gap between all-GT and matched-only accuracy shows that localization and observation coverage, not just binary classification, dominate the full-building endpoint. Because matched populations change by view, their classification percentages must not replace the fixed-denominator measure.

For the paired cruise-to-floor comparison, 2,696 building outcomes change from incorrect to correct and 261 change from correct to incorrect, a net gain of 2,435 on this development population. The same nominally closer view therefore supplies useful detail and harmful changes. These are binary ChangeOS diagnostics; the archived four-class results in Appendix B belong to a different backend and are not pooled with them. More importantly, building-level gains do not by themselves imply better task answers.

7.2. RQ2 and RQ3: Frozen online policies and task answers

Table 3 reports the complete online comparison. T1_TASK has the highest observed accuracy, 75.00%, compared with 72.29% for A0_HOLD. It uses 161 executed reobservations across 480 question-repeat episodes, compared with 419 for random, 923 for always, and 626 for raw entropy. The higher observed score and lower realized reobservation count are useful descriptive results. They are not a matched-cost comparison: policies have the same maximum cap but choose different numbers of actions.

The four prespecified paired contrasts are in Table 4. T1_TASK exceeds A0_HOLD by an observed 2.71 percentage points, but the multiplicity-adjusted ROI-cluster interval includes zero. The appropriate claim is that T1 is higher *in this final run*, while a stable advantage over holding remains uncertain. Its contrast with raw entropy is likewise uncertain. The interval against random touches zero at its lower endpoint, so it does not establish a strict positive difference under the declared interval rule. T1 is above always reobserving within the prespecified cluster analysis, but this comparison alone does not prove that its individual movements caused the gain.

Table 3: Frozen final-set online policies. Each row contains the same 160 questions under three generation repeats (480 question-repeat episodes); repeats are not independent questions. Wall time is measured software episode runtime, not flight duration. A3 and A4 are supplemental comparisons.

Policy	Correct / 480	Accuracy (%)	Reobservations	Wall time (s/episode)
A0_HOLD	347	72.29	0	17.03
A1_RANDOM	326	67.92	419	33.65
A2_ALWAYS	307	63.96	923	51.07
A3U_RAW_ENTROPY	339	70.63	626	39.61
A3_ENTROPY	341	71.04	628	38.94
A4_CONFORMAL	338	70.42	628	38.95
T1_TASK	360	75.00	161	17.92

Table 4: Prespecified T1_TASK paired accuracy contrasts. Differences are percentage points; 98.75% ROI-cluster percentile intervals adjust for the four primary comparisons. All questions and repeats from a sampled ROI remain together.

Contrast	Difference (pp)	98.75% interval (pp)
T1 minus A0_HOLD	+2.71	[−2.04, +7.50]
T1 minus A1_RANDOM	+7.08	[0.00, +14.70]
T1 minus A2_ALWAYS	+11.04	[+0.86, +21.38]
T1 minus A3U_RAW_ENTROPY	+4.38	[−4.33, +12.96]

Additional observations can alter the answer in either direction. Within their own episode histories, T1_TASK records 36 corrected and 21 harmed answers; A2_ALWAYS records 74 corrected and 132 harmed. These transition labels are not paired counterfactual effects versus A0_HOLD. They nevertheless show why an always-reobserve policy can spend the largest action budget while finishing with the lowest accuracy. A1_RANDOM executes 419 reobservations and reaches 67.92%; A3U_RAW_ENTROPY executes 626 and reaches 70.63%. Neither action count nor classifier uncertainty alone orders final task quality.

The episode traces show concrete instances in both directions. In the first generation repeat, a Hurricane Michael damage question (ID ending `damage_0001_9430`) changes from an initially incorrect “no damage” answer to the correct “damaged” answer after two T1 reobservations. A Hurricane Michael spatial question (ID ending `spatial_0092_5452`) changes from the correct “north” to the incorrect “southwest” after one. These examples describe recorded answer sequences, not isolated causal effects of the movement.

7.3. RQ4 and RQ5: Task dependence, bottlenecks, and scope of the gain

Task-type outcomes reveal uneven effects. For T1_TASK versus A0_HOLD, damage accuracy is 75.76% versus 66.67% and count accuracy is 78.29% versus 75.97%. Presence is identical at 93.18%; spatial accuracy is lower for T1, 41.38% versus 43.68%. These are descriptive, correlated subgroup summaries, not four additional adjusted tests. They are consistent with the design premise that a wide context can already suffice for presence while a more detailed target view may help damage, but the spatial result limits any universal benefit claim.

Table 5: Task evidence requirements and descriptive final accuracy. A0 and T1 use the same questions and three repeats. The evidence requirements follow the task definitions; they are not experimentally isolated causes of the accuracy differences.

Task	Evidence requirement	A0 (%)	T1 (%)
Presence	Regional coverage and positive evidence	93.18	93.18
Damage	Target identity and local damage detail	66.67	75.76
Count	ROI coverage and consistent object aggregation	75.97	78.29
Spatial	Reference geometry and nearest-target identity	43.68	41.38

Table 5 connects these outcomes to benchmark construction. Narrowing a view allocates more source pixels to a building, but it can remove other buildings needed for a count or the context needed to identify the nearest damaged target. The benchmark therefore exposes competing requirements

for detail and coverage rather than ranking every narrower view as better. The platform’s joint record of ROI coverage, predicted objects, actions, and answers makes such trade-offs inspectable; it does not by itself resolve their causal contributions.

Event outcomes are also heterogeneous. T1_TASK versus A0_HOLD is 54.86% versus 58.33% in Hurricane Michael, 84.31% versus 77.12% in the Moore tornado, and 83.06% versus 79.23% in Nepal flooding. The pooled numerical gain is not reproduced in every event, and three events cannot support an unseen-disaster generalization claim. Accuracy across T1’s three generation repeats is 75.00%, 74.38%, and 75.63%; repeat stability does not remove ROI or event uncertainty.

Development diagnostics provide limited bottleneck evidence without forming an additive oracle decomposition. A deterministic answerer using predicted structured evidence reaches 66.87% on an earlier development run. Full-ROI ground-truth evidence with a deterministic rule answerer reconstructs 160/160 development reference answers, checking the question/label pipeline rather than online performance. Their source fingerprints and interventions differ, and current-visible GT with Qwen, full-task GT with Qwen, and a full-coverage always-floor reference remain unmeasured. The final T1/A0 comparison therefore isolates neither the perception error nor the language-generation error as a unique cause.

The observed T1 operating point combines a modest numerical accuracy gain over holding with fewer executed reobservations than other active policies. Its uncertainty interval, event variation, and incomplete oracle chain favor a platform-and-mechanism interpretation rather than a demonstrated generally superior task-conditioned algorithm. A randomized common-budget sweep would be needed to estimate a genuine accuracy–cost curve; the seven realized operating points cannot supply one by rebinning episodes after execution.

8. Limitations and deployment boundary

The camera ladder crops and resamples existing orthographic imagery. It changes geographic footprint and assigned source pixels, but it cannot synthesize a new viewing angle, acquisition time, illumination condition, occluded surface, or detail below native resolution. Pre- and post-disaster channels undergo different resampling, and basemap context may come from a different date. The fixed flat-ground, nadir abstraction does not validate altitude-dependent image formation for a physical UAV.

The vehicle model uses simplified position updates without wind, attitude dynamics, acceleration limits, terrain and collision avoidance, communications, or battery discharge. Executed reobservations are measured software actions.

State-derived horizontal and vertical displacements are geometric estimates. Of 3,360 final episodes, 42 end with a fly action lacking a subsequent recorded state, so complete-motion summaries are available for 3,318. Episode wall time is measured execution time on the experiment system, not measured flight time; render, ChangeOS, controller, and Qwen component latencies were not separately recorded. The study therefore concerns imagery-grounded active observation, not operational flight performance.

The ChangeOS checkpoint is fixed across policies but has prior exposure to related xBD data; the final policy comparison does not imply event-disjoint perception training. The final questions occupy 44 ROIs in only three disaster events. All 160 final questions were independently reviewed by two annotators and adjudicated before the set was frozen. This reduces reference-answer and task-scope ambiguity, but it does not eliminate possible dataset artifacts. Answer frequencies and template families may still permit language-only shortcuts, and the present study does not include a separate language-only evaluation on the frozen final set.

The generation protocol gives three seeded repeats, but active policies can make different numbers of later Qwen calls. Their paired outcomes thus compare entire decision pipelines rather than the isolated causal effect of one movement. The seven policies share a maximum reobservation cap, not equal executed action counts. No randomized cap sweep, full module-level timing, or complete layered GT/Qwen oracle suite is available. The interval for T1_TASK versus A0_HOLD includes both a small disadvantage and a useful advantage, and one of three event directions is negative. These boundaries preclude claiming stable superiority over holding or generalization to unseen disasters from the present sample.

Archived four-class allocation and VLN component results in Appendix B use different populations and/or an earlier observation stack. They are historical diagnostics, not replications of the binary ChangeOS final experiment, and their calibration constants or performance numbers are not transferred into the main evidence chain.

9. Conclusion

DisasterClaw connects disaster records, an interactive geographic world, a task benchmark, and traceable closed-loop inspection. Paired xBD images and building annotations are anchored to map coordinates shared by the vehicle, observation footprint, and task ROI. Programmatic rules derive presence, damage, count, and spatial questions with annotation- and geometry-based answers; automated checks and dual review produce 160 final tasks over 44 ROIs in three events. A shared console and headless interface then

record how an agent’s movement changes its observation, perception evidence, and terminal answer. This data-to-world-to-benchmark construction is the principal contribution.

In 160 dual-reviewed final questions over three generation repeats, T1_TASK reaches 75.00% accuracy versus 72.29% for holding and executes fewer reobservations than the other active policies. This is an observed improvement in the tested sample, not an established stable advantage over no reobservation: the prespecified adjusted interval for the 2.71-point difference spans zero. The always-reobserve policy uses 923 actions and finishes at 63.96%, illustrating that more observation can also be harmful. Together with the development camera-ladder’s corrections and harms, these results support the platform’s role in measuring task-dependent observation value.

The benchmark makes differing evidence needs explicit: damage assessment can benefit from target detail, while counts and spatial relations also depend on regional coverage and reference geometry. The observed task and event variation supports studying those requirements separately, not recommending one observation policy universally. The next scientific steps are a randomized common-budget comparison, fuller oracle decomposition, component-level timing, independently reviewed questions across more events, and a perception checkpoint evaluated without event exposure. Claims about real flight require external action and imaging validation. Until then, Disaster-Claw provides an imagery-grounded platform and benchmark with explicitly bounded evidence, not a demonstrated general-purpose flight policy.

Declarations for author completion

Acknowledgements and funding. *To be supplied by the authors, including funder names, grant numbers, and relevant contributions. No absence of funding is asserted.*

CRedit author statement. *To be assigned and approved by the named authors.*

Declaration of competing interest. *To be confirmed by all authors.*

Data and code availability. The working package contains manuscript source, frozen P6 protocol and hashes, audited final episode logs, and analysis scripts. Access to the underlying imagery remains subject to its original terms. A public archive, license, anonymization review, and complete imagery/checkpoint package have not yet been confirmed; a final availability statement must identify the actual release and its restrictions.

Generative AI assistance. Generative AI tools were used to assist manuscript restructuring, English-language editing, reference checking, and

preparation of analysis and typesetting scripts. All scientific claims, experimental design, reported results, and final manuscript content were reviewed and approved by the authors, who take full responsibility for the work.

Appendix A. Reproducibility and provenance

Appendix A.1. Frozen final identities and audit boundary

The final protocol is stored in `cja_en/review/changeos_p6_final_protocol_20260915.json`. It records full SHA-256 values for the final task set, a review report, the frozen three-event manifest, ChangeOS weights, source fingerprint, system prompt, policy configuration, and analysis scripts. Its own SHA-256 begins `41ccbb5fed5e`; exact paths and full hashes are retained in the machine-readable protocol rather than truncated in the audit. These identities describe the local execution; they are not substitutes for a publicly recoverable image and checkpoint package.

The dual-review procedure described in Section 4 requires two independent review records and an adjudication record linked to the frozen question identifiers. The available protocol lists one review-report path and hash; it does not separately identify those three records. Their inclusion and explicit mapping to the frozen set remain requirements for a complete release. This distinction prevents the execution audit from being interpreted as independent verification of the human-review process. Review records are provenance, not model inputs.

The 12 final shard directories contain manifests, preflight receipts, `episodes.jsonl`, `results.json`, and strict per-shard audits. Their combined report is `p6_final_three_event_v1_20260915_report.json` under `runs/benchmarks/changeos_binary/`; its SHA-256 begins `9e4370325bc9`. It validates 3,360 rows with zero execution errors, exact question/configuration/repeat coverage, and declared input identities. This is a local execution and consistency audit, not an independent re-execution of the full model pipeline. The run uses a fixed ChangeOS checkpoint and a locally loaded Qwen2.5-VL-7B-Instruct model; checkpoint identity and generation settings must accompany any release rather than be inferred from an online model name.

Scene construction and rendering are in the `backend/` modules `world.py`, `geo.py`, `xbd_map.py`, `fov_ladder.py`, and `perception.py`. Inspection, reobservation, and local Qwen calls are in the same directory’s `agent_vqa.py`, `recheck.py`, and `local_qwen_v1.py`. The headless protocol is under `scripts/benchmarks/`; this directory also contains `build_roi_index.py`, `gen_agent_vqa_testset_v2.py`, and `review_agent_vqa_testset.py` for coverage indexing, task generation, and review checks. The operator interface

is under `frontend/src/`. This file map establishes software correspondence, not that every optional module was active in each policy.

The final cost audits are `runs/benchmarks/changeos_binary/p7_cost_observability_final_repeat0_20260917/report.json`, with corresponding `repeat1` and `repeat2` paths. The existing auditor was applied to four shards at a time because its duplicate key is not repeat-aware. Together the three reports cover 3,360 episodes and flag 42 incomplete terminal-motion traces. All logged recheck request, inferred allocation, execution, and episode totals agree. Search-action allocation details, component-level timing, actual flight time, and physical action mapping remain unavailable. A complete public reproduction package would additionally need the licensed source imagery, scene manifests and affine transforms, basemap availability, dependency lock, checkpoint retrieval instructions, and one replay with action/observation hashes.

Appendix B. Historical diagnostics outside the main evidence chain

Appendix B.1. Archived four-class camera and allocation study

An earlier `xview2_first` four-class pipeline evaluated 3,753 buildings in 44 ROIs. It is not the binary ChangeOS backend of the final study. Table B.1 retains its camera-footprint results and Table B.2 its offline allocation at a 0.25 building-selection fraction. The old temperature, prediction-set threshold, four-class entropy normalization, and allocation scores are not used in the main experiment. These offline selections operate on paired stored predictions, not on executable closed-loop routes or question answers.

Table B.1: Historical four-class building-level camera diagnostic on the same 3,753 evaluation buildings. Not comparable to the binary ChangeOS final question endpoint.

View	GSD (m/px)	Acc.	Macro-F1	Brier	NLL
Cruise	1.50	0.7250	0.6118	0.4814	1.3564
Mid	1.00	0.7357	0.6327	0.4669	1.3314
Floor	0.50	0.7423	0.6542	0.4616	1.3073

The archived matched-only sensitivity population contains 2,757 buildings detected at all three views (Table B.3). Because the subset is selected by detector success, it cannot replace the full 3,753-building endpoint. Its earlier exploratory ROI-bootstrap intervals, shown in Table B.4, condition on fixed scores and the observed three events; they do not demonstrate transfer to new disasters.

Table B.2: Historical four-class offline allocation at a 0.25 building-selection fraction. The label-informed row uses evaluation labels and is not deployable.

Policy	Actions	Acc.	Macro-F1	ECE	Net correct
No reobservation	0	0.7250	0.6118	0.2423	0
Random	938	0.7301	0.6241	0.2437	19
Raw entropy	938	0.7397	0.6448	0.2578	55
Calibrated entropy	938	0.7391	0.6433	0.2609	53
Expected entropy reduction	938	0.7357	0.6308	0.2551	40
APS set size	938	0.7343	0.6367	0.2549	35
Label-informed diagnostic	938	0.7759	0.6928	0.2853	191

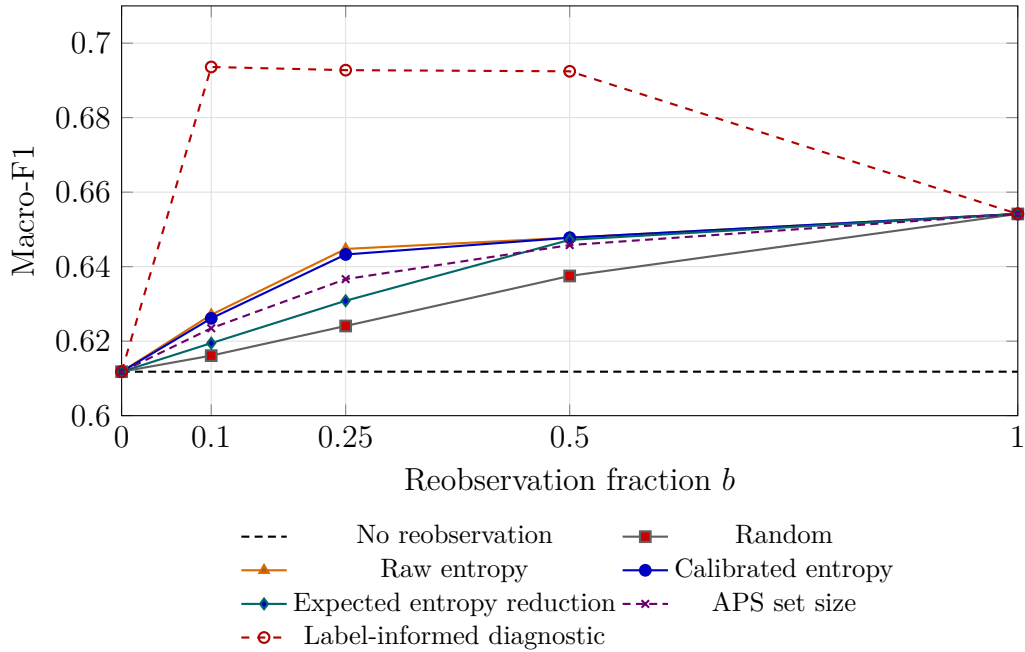


Figure B.7: Historical offline four-class allocation sweep. Selected floor predictions replace cruise predictions; this is not a common-budget online policy curve.

Table B.3: Historical four-class matched-only sensitivity analysis.

View	Accuracy	Macro-F1	Brier	NLL
Cruise	0.7918	0.7069	0.3367	0.6455
Mid	0.8052	0.7321	0.3176	0.6123
Floor	0.8099	0.7489	0.3175	0.6103

Table B.4: Historical four-class exploratory paired ROI-bootstrap differences at the 0.25 offline allocation fraction.

Paired comparison	Macro-F1 difference	95% interval
Calibrated entropy – Random	+0.0192	[+0.0097, +0.0419]
Calibrated entropy – Raw entropy	-0.0015	[-0.0093, +0.0061]

Appendix B.2. Archived VLN switch diagnostic

The earlier 200-instruction VLN switch experiment used an earlier observation stack and one recorded run per configuration. Table B.5 preserves the archived aggregate as a component diagnostic only. Enabling the recheck configuration did not cause recheck actions in that run; the table cannot establish a benefit from planning, memory, or reobservation under the current ChangeOS binary protocol.

Table B.5: Archived VLN switch diagnostic. SR is strict success within 25 m of the specified target; semSR accepts a matching damage-class building within 25 m. Recheck triggers were zero in all four configurations.

Config.	Enabled navigation components	n	SR (95% CI)	semSR	NE (m)
B0	Keyword/greedy baseline	200	0.070 [0.042, 0.114]	0.165	222.3
B1	Hierarchical semantic planning	200	0.080 [0.050, 0.126]	0.155	223.9
B2	B1 + uncertainty recheck	200	0.080 [0.050, 0.126]	0.155	223.9
B3	B2 + topological memory	200	0.080 [0.050, 0.126]	0.140	305.7

References

- [1] P. Anderson, Q. Wu, D. Teney, J. Bruce, M. Johnson, N. Sünderhauf, I. Reid, S. Gould, A. van den Hengel, Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 3674–3683.
- [2] S. Liu, H. Zhang, Y. Qi, P. Wang, Y. Zhang, Q. Wu, AerialVLN: Vision-and-language navigation for UAVs, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 15384–15394.
- [3] J. Lee, T. Miyanishi, S. Kurita, K. Sakamoto, D. Azuma, Y. Matsuo, N. Inoue, CityNav: A large-scale dataset for real-world aerial navigation, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2025, pp. 5912–5922.

- [4] Y. Gao, C. Li, Z. You, J. Liu, Z. Li, P. Chen, Q. Chen, Z. Tang, L. Wang, P. Yang, Y. Tang, Y. Tang, S. Liang, S. Zhu, Z. Xiong, Y. Su, X. Ye, J. Li, Y. Ding, D. Wang, X. Li, Z. Wang, B. Zhao, OpenFly: A comprehensive platform for aerial vision-language navigation, arXiv preprint arXiv:2502.18041v7 (2026). doi:10.48550/arXiv.2502.18041.
- [5] D. Shah, B. Osiński, B. Ichter, S. Levine, LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action, in: Proceedings of The 6th Conference on Robot Learning, Vol. 205 of Proceedings of Machine Learning Research, PMLR, 2023, pp. 492–504.
- [6] W. Zhang, C. Gao, S. Yu, R. Peng, B. Zhao, Q. Zhang, J. Cui, X. Chen, Y. Li, CityNavAgent: Aerial vision-and-language navigation with hierarchical semantic planning and global memory, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria, 2025, pp. 31292–31309. doi:10.18653/v1/2025.acl-long.1511.
- [7] Y. Gao, Z. Wang, P. Han, L. Jing, D. Wang, B. Zhao, Exploring spatial representation to enhance LLM reasoning in aerial vision-language navigation, arXiv preprint arXiv:2410.08500 (2024).
- [8] K. Li, J. Yang, L. Zhang, G. Yu, C. Yan, Y. Ding, D. Wang, N. Luo, G. Liu, X. Gao, Q. Wang, AerialClaw: An open-source framework for LLM-driven autonomous aerial agents, arXiv preprint arXiv:2606.12142 (2026). doi:10.48550/arXiv.2606.12142.
- [9] Y. Zhao, K. Xu, Z. Zhu, Y. Hu, Z. Zheng, Y. Chen, Y. Ji, C. Gao, Y. Li, J. Huang, CityEQA: A hierarchical LLM agent on embodied question answering benchmark in city space, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, 2025, pp. 12465–12480. doi:10.18653/v1/2025.emnlp-main.630.
- [10] F. Yao, et al., AeroVerse: Uav-agent benchmark suite for simulating, pre-training, finetuning, and evaluating aerospace embodied world models, arXiv preprint arXiv:2408.15511 (2024). arXiv:2408.15511.
- [11] M. Guo, et al., Bedi: A comprehensive benchmark for evaluating embodied agents on uavs, ISPRS Journal of Photogrammetry and Remote Sensing 232 (2026) 910–936. doi:10.1016/j.isprsjprs.2026.01.013.
- [12] X. Wang, et al., Towards realistic uav vision-language navigation: Platform, benchmark, and methodology, arXiv preprint arXiv:2410.07087 (2024). arXiv:2410.07087.

- [13] R. Gupta, R. Hosfelt, S. Sajeed, N. Patel, B. Goodman, J. Doshi, E. Heim, H. Choset, M. Gaston, xBD: A dataset for assessing building damage from satellite imagery, arXiv preprint arXiv:1911.09296 (2019). doi:10.48550/arXiv.1911.09296.
- [14] Z. Liu, D. Zhao, B. Yuan, RescueADI: Adaptive disaster interpretation in remote sensing images with autonomous agents, arXiv preprint arXiv:2410.13384 (2024). doi:10.48550/arXiv.2410.13384.
- [15] A. Shabbir, M. A. Munir, A. Dudhane, M. U. Sheikh, M. H. Khan, P. Fraccaro, J. B. Moreno, F. S. Khan, S. Khan, Thinkgeo: Evaluating tool-augmented agents for remote sensing tasks, arXiv preprint arXiv:2505.23752 (2025). arXiv:2505.23752.
- [16] J. Wang, et al., Disaster3: A remote sensing vision-language dataset for disaster damage assessment and response, Advances in Neural Information Processing Systems (2025). arXiv:2505.21089.
- [17] R. Bajcsy, Active perception, Proceedings of the IEEE 76 (8) (1988) 996–1005.
- [18] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. B. Tenenbaum, T. Lozano-Pérez, L. P. Kaelbling, Partially observable task and motion planning with uncertainty and risk awareness, in: Proceedings of Robotics: Science and Systems, 2024. doi:10.15607/RSS.2024.XX.118.
- [19] C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger, On calibration of modern neural networks, in: Proceedings of the 34th International Conference on Machine Learning, Vol. 70 of Proceedings of Machine Learning Research, 2017, pp. 1321–1330.
- [20] Y. Ovadia, E. Fertig, J. Ren, Z. Nado, D. Sculley, S. Nowozin, J. Dillon, B. Lakshminarayanan, J. Snoek, Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift, in: Advances in Neural Information Processing Systems, Vol. 32, 2019.
- [21] Y. Geifman, R. El-Yaniv, SelectiveNet: A deep neural network with an integrated reject option, in: Proceedings of the 36th International Conference on Machine Learning, Vol. 97 of Proceedings of Machine Learning Research, 2019, pp. 2151–2159.
- [22] A. Z. Ren, A. Dixit, A. Bodrova, S. Singh, S. Tu, N. Brown, P. Xu, L. Takayama, F. Xia, J. Varley, Z. Xu, D. Sadigh, A. Zeng, A. Majumdar, Robots that ask for help: Uncertainty alignment for large language model planners, in: Conference on Robot Learning, Vol. 229 of Proceedings of Machine Learning Research, 2023, pp. 661–682.